Programming Fundamentals: Lab 12

Lab Tasks 6-10

Name: Muhammad Farhan
Roll no: 25K-0538
Section: BCS-1K
Course: Programming Fundamentals Lab
Course Instructor: Sir Sheeraz Iqbal

Task 6:

```
#include <stdio.h>

#include <stdlib.h>


int main()

{

                    int* ptr;

                    *ptr = 5;

                    printf("\n%d", *ptr);

return 0;

}
```

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int* ptr;
    *ptr = 5;
    printf("\n%d", *ptr);
return 0;
}
```

```
5
[Finished in 0.25s]
```

Task 7:

```
#include <stdio.h>

#include <stdlib.h>


int main()

{

                int* ptr = (int*)malloc(5 * sizeof(int));

                if (ptr==NULL)

                {

                    printf("\nInitial Malloc Failed.\n");

                    return 1;

                }

                for (int i = 0; i < 5; ++i)

                {

                    ptr[i] = i;

                }

                int *ptr2 = (int*)realloc(NULL, 5*sizeof(int));

                if (ptr2==NULL)

                {

                    printf("\nRealloc Failed.\n");

                    free(ptr);

                    return 1;

                }

                printf("The 'Resized' memory address is: %d", ptr2);


                if (ptr!=ptr2)
```

{

printf("\nThe fact that this condition is true proves that ptr2 points to a completely new block of memory.");

printf("\nThis proves that realloc(NULL, size) behaves just like malloc(size) as it grabs a random piece of memory.");

}

free(ptr);

ptr = NULL;

free(ptr2);

ptr2 = NULL;

return 0;

}

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int* ptr = (int*)malloc(5 * sizeof(int));
    if (ptr==NULL)
    {
        printf("\nInitial Malloc Failed.\n");
        return 1;
    }
    for (int i = 0; i < 5; ++i)
    {
        ptr[i] = i;
    }
    int *ptr2 = (int*)realloc(NULL, 5*sizeof(int));
    if (ptr2==NULL)
    {
        printf("\nRealloc Failed.\n");
        free(ptr);
        return 1;
    }
    printf("The 'Resized' memory address is: %d", ptr2);

    if (ptr!=ptr2)
    {
        printf("\nThe fact that this condition is true proves that ptr2 points to a completely new block of memory.");
        printf("\nThis proves that realloc(NULL, size) behaves just like malloc(size) as it grabs a random piece of memory.");
    }
    free(ptr);
    ptr = NULL;
    free(ptr2);
    ptr2 = NULL;
    return 0;
}
```

```
The 'Resized' memory address is: -1620763760
The fact that this condition is true proves that ptr2 points to a completely new block of memory.
This proves that realloc(NULL, size) behaves just like malloc(size) as it grabs a random piece of memory.
[Finished in 0.41s]
```

Task 8:

```
#include <stdio.h>

#include <stdlib.h>


int main()

{

                int *arr1 = (int*)malloc(10 *sizeof(int));

                if (arr1==NULL)

                {

                    printf("\nMalloc Failed.");

                    return 1;

                }

                for (int i = 0; i < 10; ++i)

                {

                    arr1[i]=i*10;

                }


                //here we would call free() to free the memory before closing and
end the program, but we forgot.

                //now proving that the memory still exists by assinging value.

                arr1[5] = 99;

                printf("\n%d The fact that this got printed proves that the memory
still exists", arr1[5]);

                free(arr1);

return 0;

}
```

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *arr1 = (int*)malloc(10 *sizeof(int));
    if (arr1==NULL)
    {
        printf("\nMalloc Failed.");
        return 1;
    }
    for (int i = 0; i < 10; ++i)
    {
        arr1[i]=i*10;
    }

    //here we would call free() to free the memory before closing and end the program, but we forgot.
    //now proving that the memory still exists by assinging value.
    arr1[5] = 99;
    printf("\n%d The fact that this got printed proves that the memory still exists", arr1[5]);
    free(arr1);
return 0;
}
```

```
99 The fact that this got printed proves that the memory still exists
[Finished in 0.33s]
```

Task 9:

```
#include <stdio.h>

#include <stdlib.h>

int main()

{

                int* array = (int*)malloc(4);

                if (array==NULL)

                {

                    printf("\nMalloc Failed.");

                    return 1;

                }

                for (int i = 0; i < 4; ++i)

                {

                    printf("\nEnter Integer#%d of the array: ", i+1);

                    scanf("%d", &array[i]);

                }


                for (int i = 0; i < 4; ++i)

                {

                    printf("\nInteger#%d of the array: %d", i+1, array[i]);

                }


                free(array);

                return 0;

}
```

```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int* array = (int*)malloc(4);
    if (array==NULL)
    {
        printf("\nMalloc Failed.");
        return 1;
    }
    for (int i = 0; i < 4; ++i)
    {
        printf("\nEnter Integer#%d of the array: ", i+1);
        scanf("%d", &array[i]);
    }

    for (int i = 0; i < 4; ++i)
    {
        printf("\nInteger#%d of the array: %d", i+1, array[i]);
    }

    free(array);
    return 0;
}
```

```
Enter Integer#2 of the array: 13

Enter Integer#3 of the array: 14

Enter Integer#4 of the array: 15

Integer#1 of the array: 12
Integer#2 of the array: 13
Integer#3 of the array: 14
Integer#4 of the array: 15
[Finished in 10.64s]
```

Task 10:

```
#include <stdio.h>

#include <stdlib.h>


int main()

{

                    int* array = (int*)malloc(2*sizeof(int));

                    if (array == NULL)

                    {

                        printf("\nMalloc Failed.");

                        return 1;

                    }

                    for (int i = 0; i < 2; ++i)

                    {

                        printf("\nEnter Integer#%d of the array: ", i+1);

                        scanf("%d", &array[i]);

                    }


int* temp = (int*)realloc(array, 4*sizeof(int));

if (temp == NULL)

{

                    printf("\nRealloc Failed.");

                    return 1;

}


free(array);
```

```
for (int i = 0; i < 4; ++i)
                    {
                        printf("\nInteger#%d of the array: %d", i+1, array[i]);
                    }

free(temp);
return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int* array = (int*)malloc(2*sizeof(int));
    if (array == NULL)
    {
        printf("\nMalloc Failed.");
        return 1;
    }
    for (int i = 0; i < 2; ++i)
    {
        printf("\nEnter Integer#%d of the array: ", i+1);
        scanf("%d", &array[i]);
    }

int* temp = (int*)realloc(array, 4*sizeof(int));
if (temp == NULL)
{
    printf("\nRealloc Failed.");
    return 1;
}

free(array);

for (int i = 0; i < 4; ++i)
    {
        printf("\nInteger#%d of the array: %d", i+1, array[i]);
    }

free(temp);
return 0;
}
```

```
Enter Integer#1 of the array: 15

Enter Integer#2 of the array: 45

Integer#1 of the array: -398958208
Integer#2 of the array: 632
Integer#3 of the array: -398982832
Integer#4 of the array: 632
[Finished in 3.88s with exit code 3221226356]
```

My Github account: https://github.com/MF4rhan

Unrelated Note: The Text Editor used is “Sublime Text”. Rather than Dev C++ or VS Code.

Its similar to Dev C++ but very modern, updated and much faster, its also known as the fastest text editor in the entire market. Its specialty is that it can even open huge files and codes almost instantly and without lagging. It doesn’t write the whole syntax or functions for you like VS Code does.